

关于2D-RoPE，我们首先计算角度  $\theta$ ，和一维RoPE相同，假设 embedding 维度仍为  $d$ ，则对于位置  $i$ ， $\theta$  的计算公式：

$$\theta_i = 10000^{-2i/d}$$

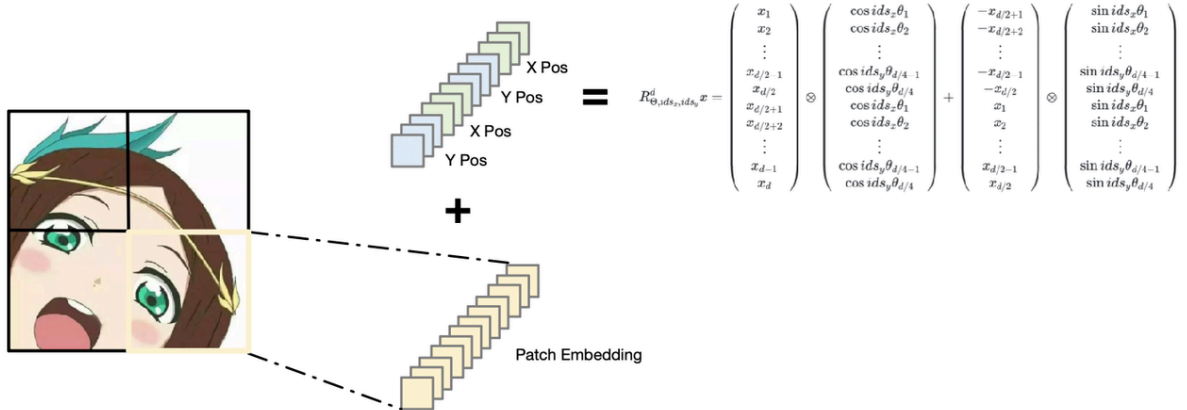
对于二维情况的旋转矩阵，我们可以对位置  $(x, y)$  的 embedding 进行以下操作，在 embedding 维度分为四个一组， $x[0], x[1]$  乘矩阵：

$$\begin{bmatrix} \cos x \theta_0 & -\sin x \theta_0 \\ \sin x \theta_0 & \cos x \theta_0 \end{bmatrix}$$

$$x[2], x[3] \text{ 则乘矩阵 } \begin{bmatrix} \cos y \theta_0 & -\sin y \theta_0 \\ \sin y \theta_0 & \cos y \theta_0 \end{bmatrix}$$

$$\text{等价于 embedding 每四个一组，乘矩阵：} \mathcal{R}_{x,y} = \begin{pmatrix} \cos x \theta & -\sin x \theta & 0 & 0 \\ \sin x \theta & \cos x \theta & 0 & 0 \\ 0 & 0 & \cos y \theta & -\sin y \theta \\ 0 & 0 & \sin y \theta & \cos y \theta \end{pmatrix}$$

其实回答到这里，就可以敷衍面试官了，但是其实在 [Qwen2-VL](#)、[Qwen2.5-VL](#) 的实现并不是和上面严格对应的，下面我们来仔细看看：



$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d/2-1} \\ x_{d/2} \\ x_{d/2+1} \\ x_{d/2+2} \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos i d_s \theta_1 & \cos i d_s \theta_2 & \vdots & \cos i d_s \theta_{d/4-1} \\ \cos i d_s \theta_1 & \cos i d_s \theta_2 & \vdots & \cos i d_s \theta_{d/4} \\ \cos i d_s \theta_1 & \cos i d_s \theta_2 & \vdots & \cos i d_s \theta_{d/4} \\ \cos i d_s \theta_1 & \cos i d_s \theta_2 & \vdots & \cos i d_s \theta_{d/4} \end{pmatrix} + \begin{pmatrix} -x_{d/2+1} \\ -x_{d/2+2} \\ \vdots \\ -x_{d/2-1} \\ x_1 \\ x_2 \\ \vdots \\ x_{d/2-1} \\ x_{d/2} \end{pmatrix} \otimes \begin{pmatrix} \sin i d_s \theta_1 & \sin i d_s \theta_2 & \vdots & \sin i d_s \theta_{d/4-1} \\ \sin i d_s \theta_1 & \sin i d_s \theta_2 & \vdots & \sin i d_s \theta_{d/4} \\ \sin i d_s \theta_1 & \sin i d_s \theta_2 & \vdots & \sin i d_s \theta_{d/4} \\ \sin i d_s \theta_1 & \sin i d_s \theta_2 & \vdots & \sin i d_s \theta_{d/4} \end{pmatrix}$$

首先根据以上的公式，可以  $\theta$  的序列编码最大到  $d/4$ （而1D-RoPE最大到  $d/2$ ），这是因为1D-RoPE是embedding维度两个一组，而2D-RoPE是4个一组，[Qwen2-VL](#)的快速计算实现如下：

$$\begin{aligned} R_{\theta, i d_s x, i d_s y}^d x = & \begin{pmatrix} x_1 & x_2 & \dots & x_{d/2-1} \\ x_{d/2} & x_{d/2+1} & \dots & x_{d-1} \end{pmatrix} \\ & \otimes \begin{pmatrix} \cos i d_s \theta_1 & \cos i d_s \theta_2 & \dots & \cos i d_s \theta_{d/4-1} \\ \cos i d_s \theta_1 & \cos i d_s \theta_2 & \dots & \cos i d_s \theta_{d/4} \\ \cos i d_s \theta_1 & \cos i d_s \theta_2 & \dots & \cos i d_s \theta_{d/4} \\ \cos i d_s \theta_1 & \cos i d_s \theta_2 & \dots & \cos i d_s \theta_{d/4} \end{pmatrix} \\ & + \begin{pmatrix} -x_{d/2+1} & -x_{d/2+2} & \dots & -x_{d/2-1} \\ -x_{d/2} & -x_{d/2+1} & \dots & -x_{d/2-1} \end{pmatrix} \\ & \otimes \begin{pmatrix} \sin i d_s \theta_1 & \sin i d_s \theta_2 & \dots & \sin i d_s \theta_{d/4-1} \\ \sin i d_s \theta_1 & \sin i d_s \theta_2 & \dots & \sin i d_s \theta_{d/4} \\ \sin i d_s \theta_1 & \sin i d_s \theta_2 & \dots & \sin i d_s \theta_{d/4} \\ \sin i d_s \theta_1 & \sin i d_s \theta_2 & \dots & \sin i d_s \theta_{d/4} \end{pmatrix} \end{aligned}$$

仔细观察发现，其实和上面的形式不一致，具体在代码层面怎么实现的呢：

```
def rot_pos_emb(self, grid_thw):
    ...
    得到每个patch位置的2D-位置编码的正余弦()内的角度信息，然后再对xy方向进行flatten
    ...
    pos_ids = []
    for t, h, w in grid_thw:
        # [h, w]
        hpos_ids = torch.arange(h).unsqueeze(1).expand(-1, w)
        # [h//spatial_merge_size, spatial_merge_size, w//spatial_merge_size, spatial_merge_size], spatial_merge_size=2
        hpos_ids = hpos_ids.reshape(
            h // self.spatial_merge_size,
            self.spatial_merge_size,
            w // self.spatial_merge_size,
            self.spatial_merge_size,
        )
        # [h//spatial merge size, w//spatial_merge_size, spatial_merge_size, spatial_merge_size]
        hpos_ids = hpos_ids.permute(0, 2, 1, 3)
        # [h*w]
        hpos_ids = hpos_ids.flatten()

        wpos_ids = torch.arange(w).unsqueeze(0).expand(h, -1)
        wpos_ids = wpos_ids.reshape(
            h // self.spatial_merge_size,
            self.spatial_merge_size,
            w // self.spatial_merge_size,
            self.spatial_merge_size,
        )
        wpos_ids = wpos_ids.permute(0, 2, 1, 3)
        wpos_ids = wpos_ids.flatten()
        # 注意这里t维度是直接复制的，意味着在视觉编码的时候，没有考虑时间(帧率)之间的区别
        pos_ids.append(torch.stack([hpos_ids, wpos_ids], dim=-1).repeat(t, 1)) # [t, h*w, 2]
        # [n*h*w, 2], 每个patch的(x,y)位置,time维度concat
    pos_ids = torch.cat(pos_ids, dim=0)
    max_grid_size = grid_thw[:, 1:].max()
```

```

# [max_grid_size, head_dim//4], x和y每个方向最大的就是\theta_{d//4}
rotary_pos_emb_full = self.rotary_pos_emb(max_grid_size)
# 分别提取每个patch的x,y的embedding角度信息，并且concat
# [nhw, 2, head_dim//4] -> [nhw, head_dim//2]
rotary_pos_emb = rotary_pos_emb_full[pos_ids].flatten(1)
return rotary_pos_emb

# 其中self.rotary pos emb定义如下，注意初始化时候dim=hidden_dim//2:
class VisionRotaryEmbedding(nn.Module):
    def __init__(self, dim: int, theta: float = 10000.0) -> None:
        super().__init__()
        # 就是  $1 / (10000^{\{2i/d\}})$ 
        inv_freq = 1.0 / (theta ** (torch.arange(0, dim, 2, dtype=torch.float) / dim))
        self.register_buffer("inv_freq", inv_freq, persistent=False)

    def forward(self, seqlen: int) -> torch.Tensor:
        # [seqlen]
        seq = torch.arange(seqlen, device=self.inv_freq.device, dtype=self.inv_freq.dtype)
        # [seqlen, dim//4], 对应每个m个对应的位置编码正余弦里面的数  $m / (10000^{\{2i/dim\}})$ 
        freqs = torch.outer(seq, self.inv_freq) # 外积
        return freqs

```

首先注意一个细节，在Qwen2VisionTransformerPretrainedModel.forward方法中，pos\_emb在hidden\_dim方向进行了一次复制，和我们上面的公式一致，前hidden\_dim//2和后hidden\_dim//2是一致的：

```

def forward(self, hidden_states: torch.Tensor, grid_thw: torch.Tensor) -> torch.Tensor:
    hidden_states = self.patch_embed(hidden_states)
    rotary_pos_emb = self.rot_pos_emb(grid_thw)
    emb = torch.cat((rotary_pos_emb, rotary_pos_emb), dim=-1)
    position_embeddings = (emb.cos(), emb.sin())
    ...

```

之后就是直接使用  $q\_embed = (q * \cos) + (\text{rotate\_half}(q) * \sin)$  进行计算即可：

```

def apply_rotary_pos_emb_vision(
    q: torch.Tensor, k: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor
) -> Tuple[torch.Tensor, torch.Tensor]:
    orig_q_dtype = q.dtype
    orig_k_dtype = k.dtype
    q, k = q.float(), k.float() # [b, seq_len, num_head, dim]
    cos, sin = cos.unsqueeze(-2).float(), sin.unsqueeze(-2).float()
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    q_embed = q_embed.to(orig_q_dtype)
    k_embed = k_embed.to(orig_k_dtype)
    return q_embed, k_embed

# 其中rotate_half是将[x1,...,x_{d/2},x_{d/2+1},...,x_d]变为[-x_{d/2+1},...,-x_d,x1,...,x_{d/2}]
def rotate_half(x):
    """Rotates half the hidden dims of the input."""
    x1 = x[..., : x.shape[-1] // 2]
    x2 = x[..., x.shape[-1] // 2 :]
    return torch.cat((-x2, x1), dim=-1)

```

拓展阅读：2D-RoPE在21年的文章《[二维位置的旋转式位置编码](#)》已经有较好的推导，但是和[Qwen2-VL](#)实现不一致！